concurrently, specific functions and objects must be used to manage the different threads.

1.4.1. Hello, Concurrent World

Let's start with a classic example: a program to print "Hello World." A really simple Hello, World program that runs in a single thread is shown here, to serve as a baseline when we move to multiple threads:

```
#include <iostream>

int main()
{
    std::cout<<"Hello World\n";
}
```

All this program does is write "Hello World" to the standard output stream. Let's compare it to the simple Hello, Concurrent World program shown in the following listing, which starts a separate thread to display the message.

Listing 1.1. A simple Hello, Concurrent World program

```
#include <iostream>
#include <thread>                    ❶

void hello()                         ❷
{
    std::cout<<"Hello Concurrent World\n";
}

int main()
{
    std::thread t(hello);            ❸
    t.join();                        ❹
}
```

The first difference is the extra `#include <thread>` ❶. The declarations for the multithreading support in the Standard C++ Library are in new headers: the functions and classes for managing threads are declared in `<thread>`, whereas those for protecting shared data are declared in other headers.